3Laws

Release latest

Thomas Gurriet (tgurriet@3laws.io)

Feb 28, 2024

1 Introduction **3**
1.1 ROS dependencies . . . 3
1.2 RealTime Assurance Module (RAM). . . . 3
1.3 Robot Diagnostic Module (RDM) . . . 4
1.4 Configuration. . . . 4

2 Installation **5**
2.1 Quick Installation . . . 5
2.2 Non Interactive Installation . . . 5

3 Quick Start **7**

4 Control Panel **9**
4.1 Launch the Control Panel . . . 9
4.2 Use the Control Panel . . . 9

5 Visualization **13**
5.1 Dashboards . . . 13
5.2 Navigation Tools . . . 13
5.3 Incident Management . . . 14

Table of Contents:

Chapter 1

Introduction

The 3Laws Robotics supervisor is a software package that provides a control filter to insure safety without compromising performance. In addition, the supervisor contains a diagnostic module to monitor relevant metrics for safety and performance.

In order to make the configuration and the usage of the supervisor easier, we provide a control panel, that will guide you through the configuration of the supervisor and help you visualize the supervisor's safety filter in action.

Installation:

- Installation

Quick Start:

- Quick Start

1.1 ROS dependencies

The 3Laws Robotics supervisor is designed to work with ROS. The supervisor is a ROS node that subscribes to the robot's state, perception and control commands, and publishes the filtered control commands to the robot's actuators. Also the supervisor can subscribes to the robot's sensors and planning topics and publishes the diagnostic information.

Multiple ROS version are supported. Here is a list of the tested versions:

Ubuntu Distribution	ROS1 version	ROS2 version
22.04	N/A	Humble/Iron
20.04	Noetic	Galactic/Foxy
18.04	Melodic	N/A

1.2 RealTime Assurance Module (RAM)

The RealTime Assurance Module (RAM) is a filter at the control level. It is designed to ensure that the robot's control commands are safe and do not violate any safety constraints. Based on formal mathematical proven methods, the RAM is able to guarantee that the robot will not violate any safety constraints, while still allowing the robot to perform at its best.

This ability can be used to safely develop your robot's control and planning algorithms without the need to worry about safety. With the supervisor, focus on your core value and let the supervisor take care of the safety.

The RAM needs a basics kinematic and dynamic model of the robot and safety constraints on state in

order to perform the right filtering. 3LawsRobotics knows that it is not always easy to describe a model of the robot which is why the supervisor is designed to be easily configurable and adaptable to the robot's model.

1.3 Robot Diagnostic Module (RDM)

The Robot Diagnostic Module (RDM) is a diagnostic tool that monitors and compute metrics about the robot health and safety. The computed metrics are published on ROS topics and can be used to monitor the robot's health and safety in real-time but can also be send to a data base for further analysis. 3LawsRobotics has developed a set of Dashboards to help developers quickly diagnose issues the robot might have. These Dashboards are made using Grafana.

1.4 Configuration

The Supervisor require a bit of configuration to work properly. Knowing that this process can be a bit complex, 3LawsRobotics created a control panel to help you configure the supervisor.

The control panel is a web application that will guide you through the configuration of the supervisor. It will ask you for the robot's model, the safety constraints and the control commands topics. Also, the control panel will help you visualize the supervisor's safety filter in action.

In order to successfully identify the user, the configuration requires a token that 3LawsRobotics will provide you with.

Chapter 2

Installation

The Supervisor is distributed as a debian package. A package is available for each of the following configuration

Ubuntu Distribution	ROS1 version	ROS2 version	Architecture
22.04	N/A	Humble/Iron	amd64/arm64v8
20.04	Noetic	Galactic/Foxy	amd64/arm64v8
18.04	Melodic	N/A	amd64/arm64v8

Table of Contents

2.1 Quick Installation 5
2.2 Non Interactive Installation 5

2.1 Quick Installation

Copy and paste the following command in a terminal to install the Supervisor:

```
bash <(curl
https://raw.githubusercontent.com/3LawsRobotics/3laws/master/rdm/install.sh)
```

This will install the Supervisor for the current distribution and architecture and the control panel.

At the end of the installation, the control panel will be started and you will be able to start the configuration of the supervisor. To do so open a web browser and go to the following address: http://localhost:8000

2.2 Non Interactive Installation

If you want to install a specific package in a non interactive mode, here is the process:

- Download the script:

```
wget https://raw.githubusercontent.com/3LawsRobotics/3laws/master/rdm/install.sh
```

- Make it executable

```
chmod +x install.sh
```

- Run it with your arguments

```
sudo ./install.sh [-hyf] [-r <ROS_DISTRO>] [-a <ARCH>] [-v <UBUNTU_VERSION>]
```

if `-yf -r <ROS_DISTRO> -a <ARCH> -v <UBUNTU_VERSION>` specified, the script is non interactive

Chapter 3

Quick Start

Before starting the supervisor be sure to have your ROS environment correctly set up and sourced.

```
source /opt/ros/<DISTRO>/setup.sh
```

To launch the Supervisor you can use the following command:

```
ros2 launch lll_rdm rdm.launch.py
```

You can add the following command line options:

- **use_sim_time**: <Boolean, default: false> Specify if the RDM use the ros time or the machine time
- **config_filename**: <String> Path to the config file if not in the */opt/3lawsRoboticsInc/robot_diagnostic_module/config* folder.
- **robot_id**: <String> name of your robot
- **log_level**: <trace | debug | info | error, default: info> Log level of the RDM
- **dry_run**: <Boolean> Start the RDM, check the config file and turn off without error if everything has been launched correctly

By default the supervisor will use the ros time and the config file in /opt/3lawsRoboticsInc/robot_diagnostic_module/config/rdm_config.yaml. You can change this default folder by setting the environment variable *LLL_CONFIG_FOLDER* to the desired path.

Chapter 4

Control Panel

The **Control Panel of 3LawsRobotics** is a helper tool to help the configuration of the 3Laws Robotics supervisor. It provides a web interface to configure the 3Laws Robotics supervisor and visualize the robot's state. Currently in alpha version, it runs on your local network and can be accessed from any device with a web browser. The backend is written in Python and uses the CGI protocol (Common Gateway Interface) to communicate with the web interface.

In order to get your account setup and use freely this RDM, please contact support@3lawsrobotics.com

This product is still at a alpha development stage. New features are developed every month and some breaking changes can still happened. The 3LawsRobotics Team is available to answer your question.

For now only a ROS1, ROS2 interfaces are configurable via this control panel.

4.1 Launch the Control Panel

The control panel is installed as a systemd service. To start the control panel, use the following command:

```
sudo systemctl start lll_control_panel
```

and to stop it:

```
sudo systemctl strop lll_control_panel
```

Also you can start it manually with the following command:

```
python3 /opt/3lawsRoboticsInc/control_panel/server.py
```

You can add the following command line options:

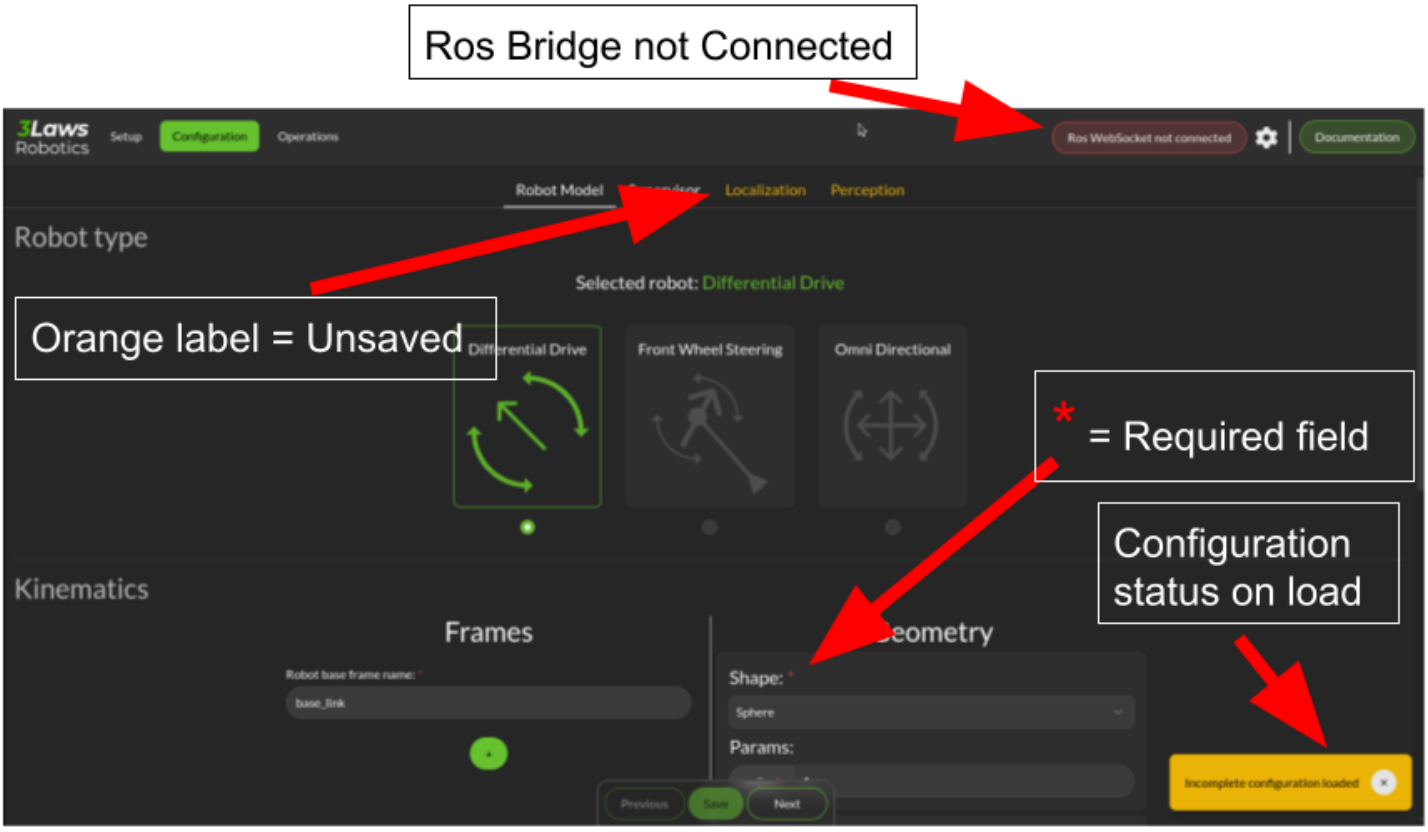
- **-p or –port**: <number, default: 8000> Specify the port to use

You can specify the emplacement of the configuration file by setting the environment variable *LLL_CONFIG_FOLDER*

4.2 Use the Control Panel

The control panel is meant to be user friendly and safe explanatory. However some detail might be useful to know.

Here is a picture of the control panel:

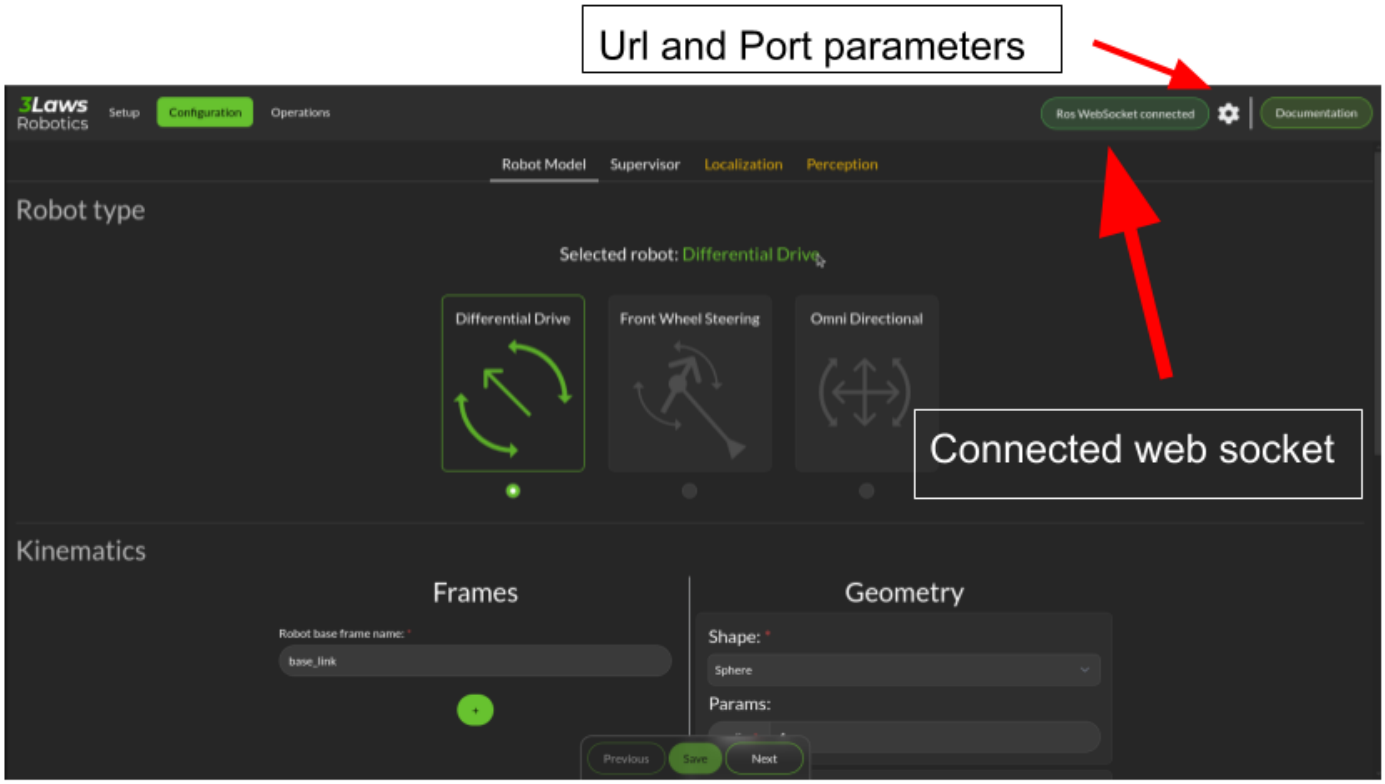


The control panel is divided in 3 parts:

- The Navigation bar, where you can find the main pages, a Ros Bridge WebSocket status indicator and a WebSocket parameter button.
- The main page, where you can find the configuration form and the control buttons.
- The footer, where you can find the version number and the 3LawsRobotics website link and a link to the update pages.

In order to use this control panel at its best, you need to configure the Ros Bridge WebSocket parameters. To do so, click on the WebSocket parameter button in the navigation bar and fill the form with the correct parameters.

By default, the control panel is configured to connect to a Ros Bridge WebSocket running on the same machine, on the port 9090. More information on the Ros Bridge WebSocket can be found following this link: Ros Bridge WebSocket.



A successful connection to the Ros Bridge WebSocket will be indicated by a green status indicator in the navigation bar.

Chapter 5

Visualization

5.1 Dashboards

In order to visualize your monitoring data. 3LawsRobotics provides a Grafana dashboard. You can access it by opening your web browser and navigating to https://3laws.app. Your credentials will be given to you by your 3LawsRobotics representative.

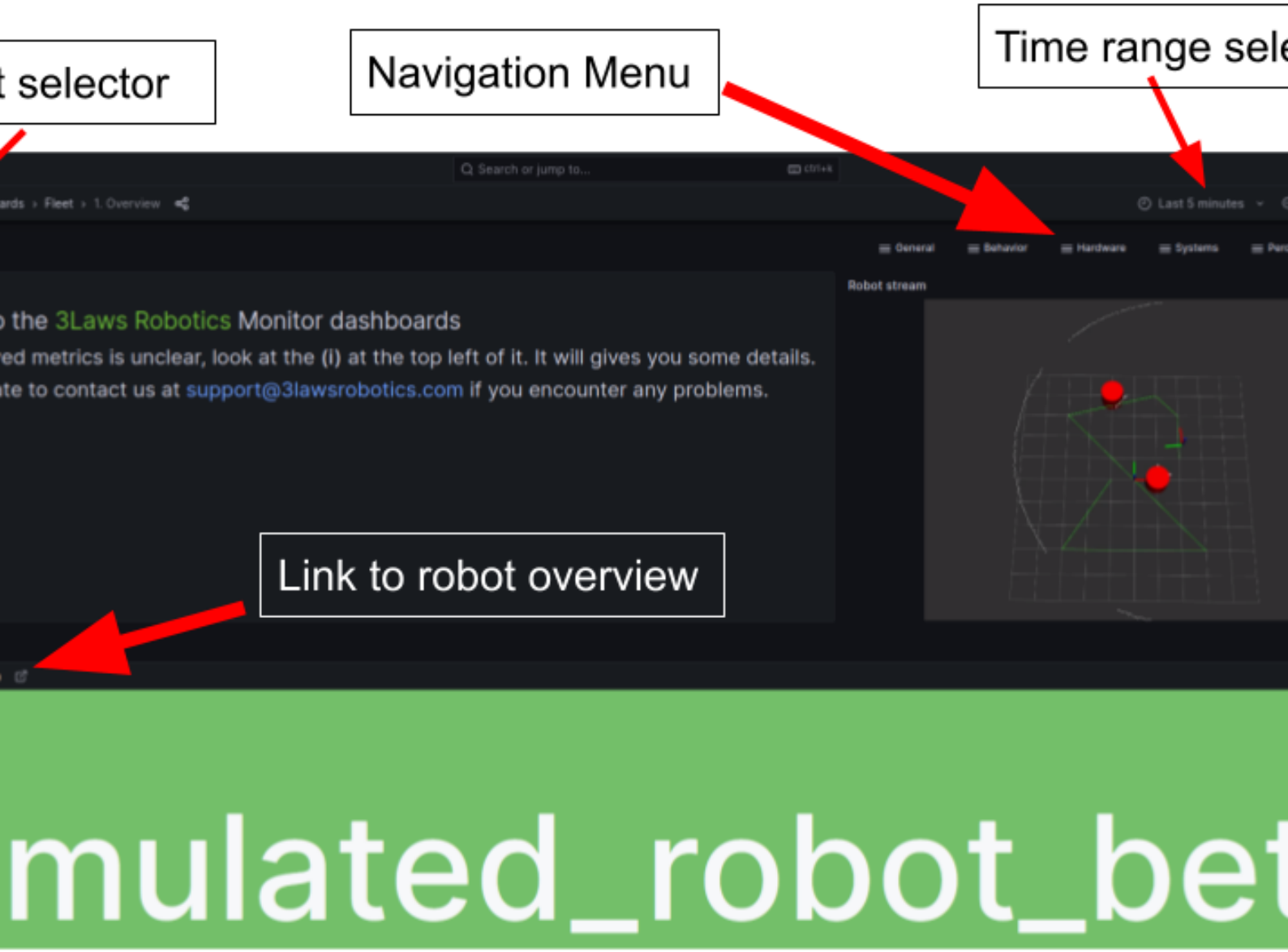
This dashboards are tailored to quick diagnostic. This means that the workflow will help you dive into the data that is relevant to your current problems. The dashboards are organized in the following way:

- **General**: This dashboard displays your robots that are currently monitored and their current state. It will also gives you a list of the latest events that have been recorded by the monitoring system.
- **Behavior**: This dashboard gives you insights into the behavior of your robot, like his computed safety score, your distance to obstacle, etc.
- **Hardware**: This dashboard expose the hardware health of the robot. It let you quickly see if the current physical behavior of the robot is coherent with your model or if some degradation is happening.
- **Systems**: This is the most important dashboard. It gives information about the system of the robot like signals delay, computer usage and Node status.
- **Perception**: This dashboard make it easy to diagnose a sensors issue like obstructed sensor or noisy sensor.
- **Control**: Finally this dashboard allows you to observe the good tracking and feasibility of the control commands or the planning algorithms

Each dashboard is composed of several panels with always an overview that helps you to quickly see if everything is fine and then more detailed panels that will help you to precisely diagnose the problem.

5.2 Navigation Tools

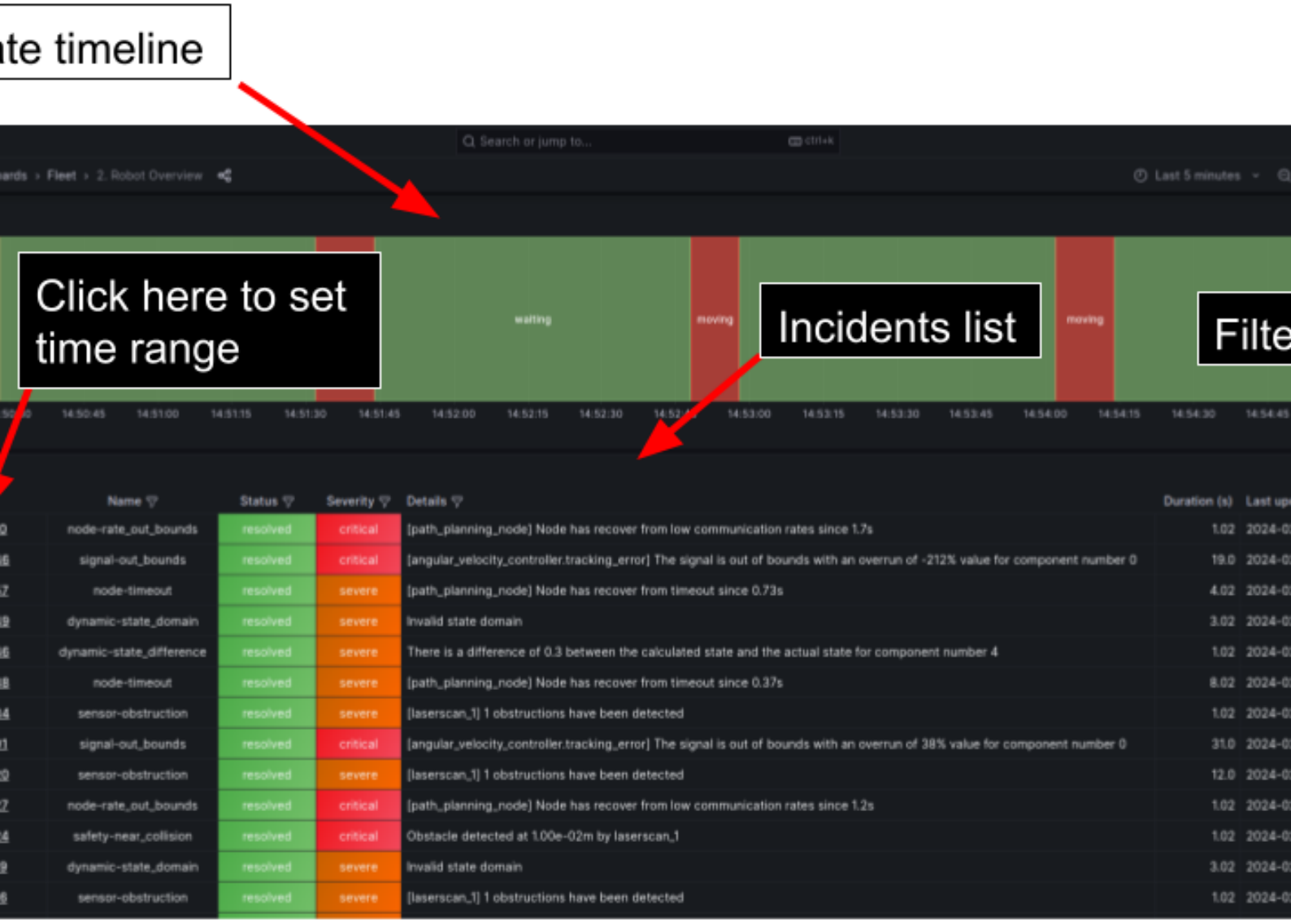
The dashboards are composed of several navigation tools that will help you to quickly diagnose a problem. Here is a screenshot of a dashboard with the navigation tools highlighted:



- **Time Range**: This tool allows you to select the time range of the data that you want to visualize. You can select a predefined range or a custom range.
- **Refresh**: This tool allows you to refresh the data displayed in the dashboard. You can select the refresh interval or manually refresh the data.

5.3 Incident Management

The Dashboards display a list of incident, you can use it to navigate directly to the right detailed dashboard. The timestamp of the incident is clickable and will bring to the right detailed dashboard at the time of the incident.


te timeline
Click here to set time range
Incidents list
Filte
Search or jump to...
Fleet
2. Robot Overview
Last 5 minutes
waiting
moving
moving
Name
Status
Severity
Details
Duration (s)
node-rate_out_bounds
resolved
critical
[path_planning_node] Node has recover from low communication rates since 1.7s
signal-out_bounds
resolved
critical
[angular_velocity_controller.tracking_error] The signal is out of bounds with an overrun of -212% value for component number 0
node-timeout
resolved
severe
[path_planning_node] Node has recover from timeout since 0.73s
dynamic-state_domain
resolved
severe
Invalid state domain
dynamic-state_difference
resolved
severe
There is a difference of 0.3 between the calculated state and the actual state for component number 4
node-timeout
resolved
severe
[path_planning_node] Node has recover from timeout since 0.37s
sensor-obstruction
resolved
severe
[laserscan_1] 1 obstructions have been detected
signal-out_bounds
resolved
critical
[angular_velocity_controller.tracking_error] The signal is out of bounds with an overrun of 38% value for component number 0
sensor-obstruction
resolved
severe
[laserscan_1] 1 obstructions have been detected
node-rate_out_bounds
resolved
critical
[path_planning_node] Node has recover from low communication rates since 1.2s
safety-near_collision
resolved
critical
Obstacle detected at 1.00e-02m by laserscan_1
dynamic-state_domain
resolved
severe
Invalid state domain
sensor-obstruction
resolved
severe
[laserscan_1] 1 obstructions have been detected